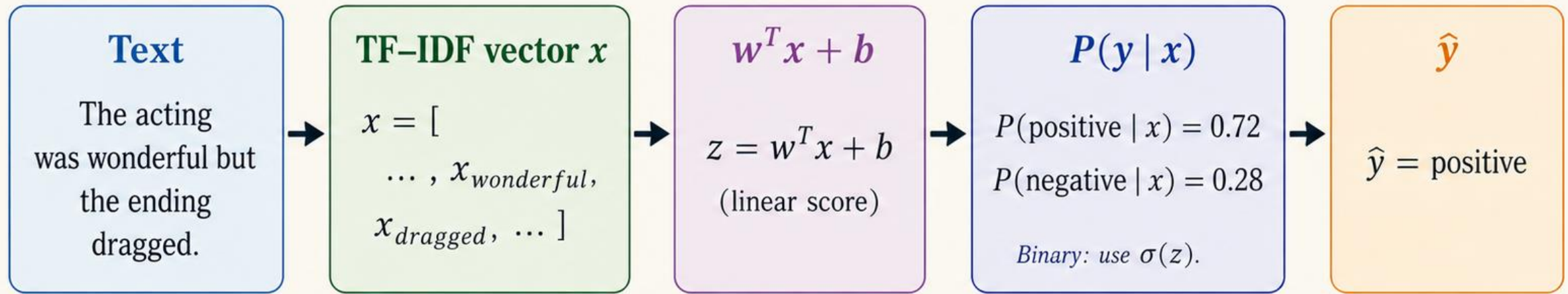


Text Classification as Probabilistic Prediction

Probabilistic label prediction from TF-IDF features.



Binary: positive / negative

Multinomial: sports / politics / business / ...

For multiple classes, replace $\sigma(z)$ by $\text{softmax}(Wx + b)$.

One text is represented as a feature vector; classification becomes probability assignment over labels.

Which words pushed the probability upward, and which pushed it down?

One Email Is One Document

TF-IDF represents each email using corpus-level document frequencies.



Term frequency: computed inside this email.

Document frequency: computed across the training-email corpus.

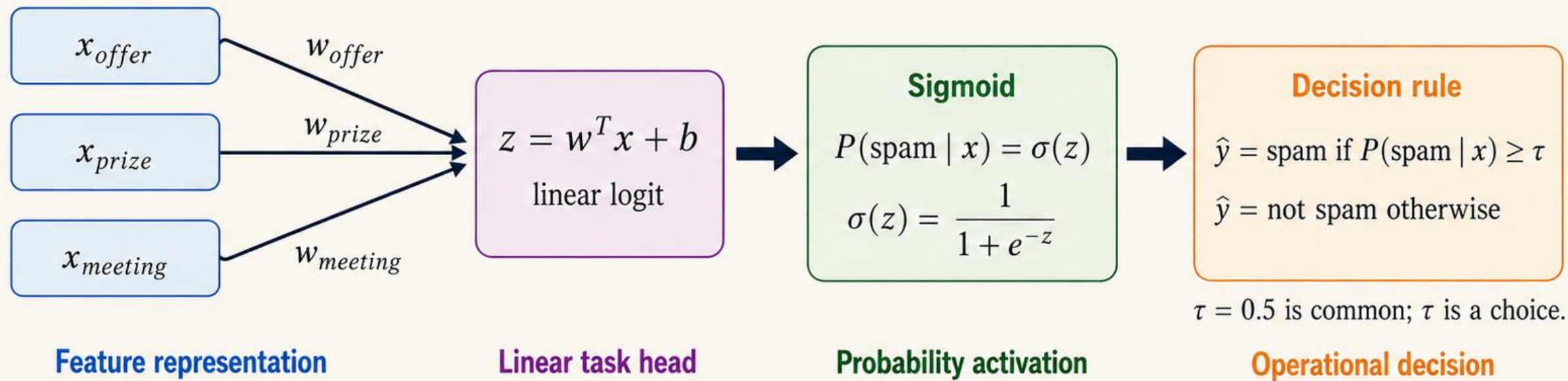
Vocabulary and IDF: fitted using training data only.

Once the email is a vector, what computation turns it into a spam probability?

Logistic Regression Is a Linear Task Head

A weighted sum of the input features becomes a class probability.

Feature representation x

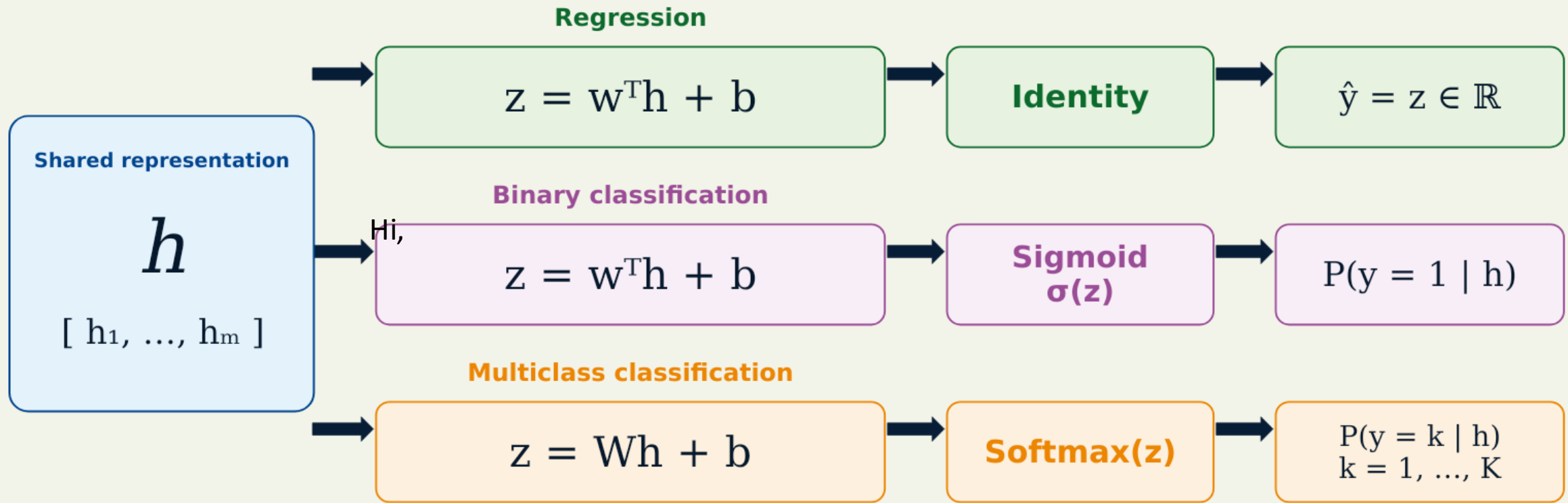


The task head is linear in the supplied representation.

How should the final layer change for a real-valued output or several classes?

Different Tasks Require Different Output Heads

The representation can be shared; the output dimension and activation follow the task.

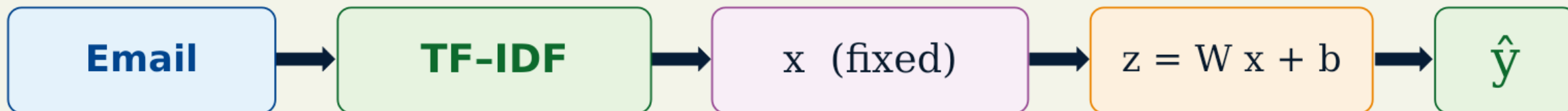


If these heads are simple, where does the model learn complex structure?

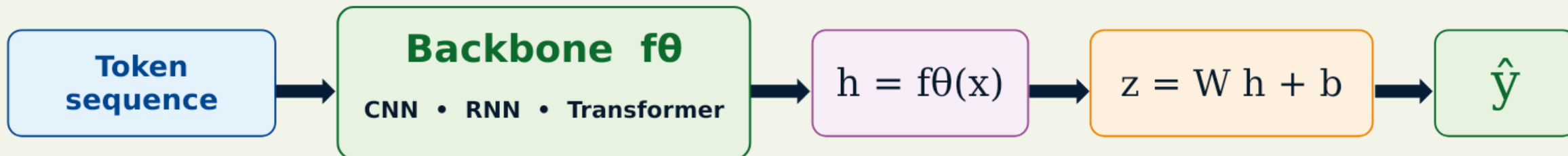
The Backbone Learns the Representation

A backbone constructs h ; the task head maps h into the required prediction space.

Fixed representation



Learned representation



Learns what information h carries

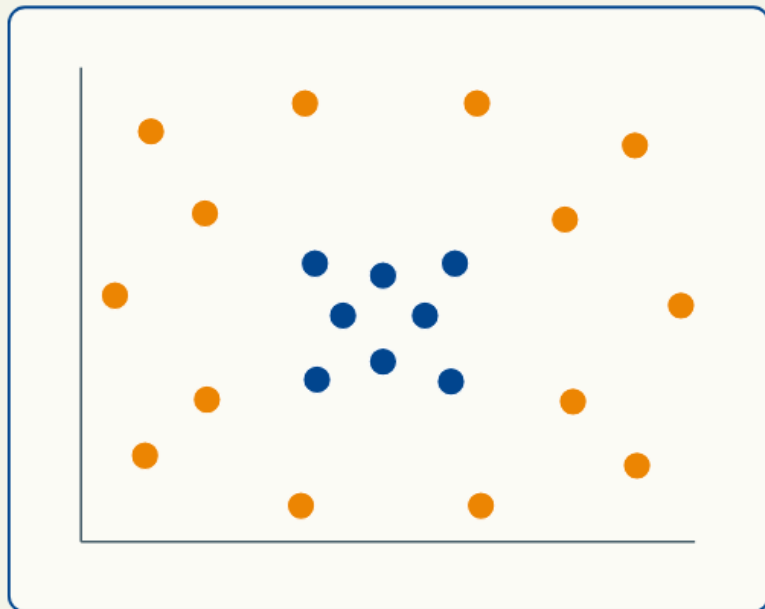
Maps h to the task output

Can a linear head solve every task once a representation is available?

Linear Separability Depends on the Representation

A nonlinear transformation can expose a distinction that a linear head can use.

Original feature space x



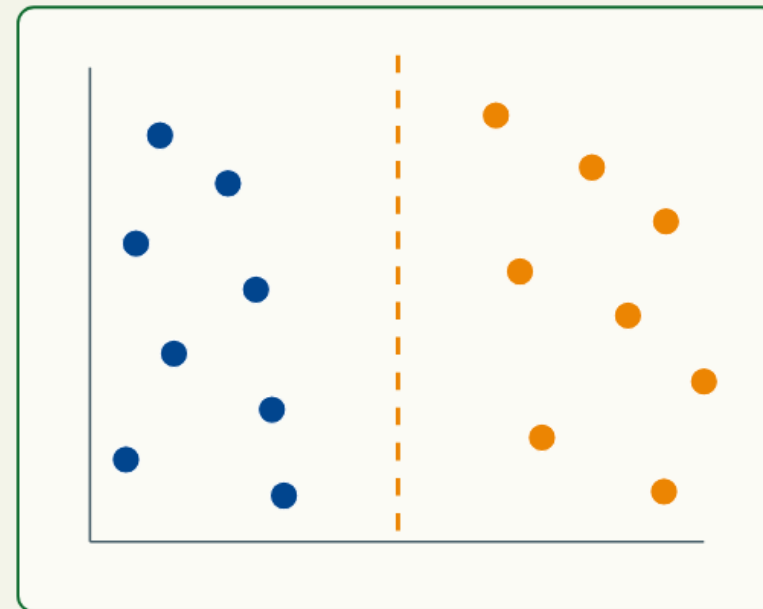
No single linear boundary

$$h = f\theta(x)$$



nonlinear
transformation

Learned representation h



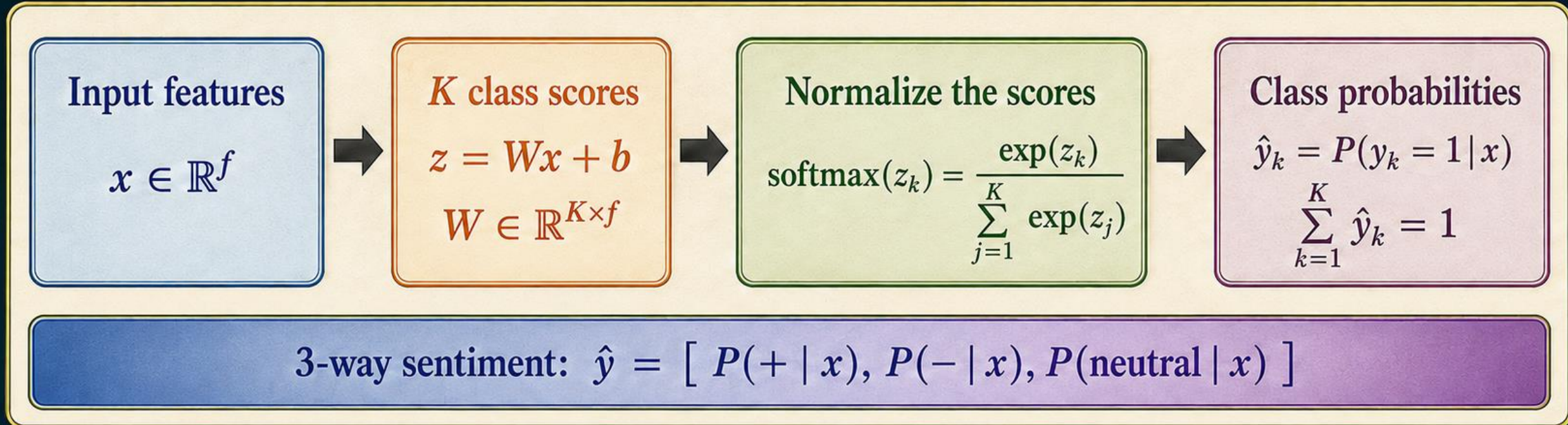
Linear boundary $z = 0$

$$z = w^T h + b$$

If backbone and head are trained together, is their separation still meaningful?

Multinomial Logistic Regression Predicts One of K Classes

Each class gets its own score; softmax converts the K scores into a probability distribution.



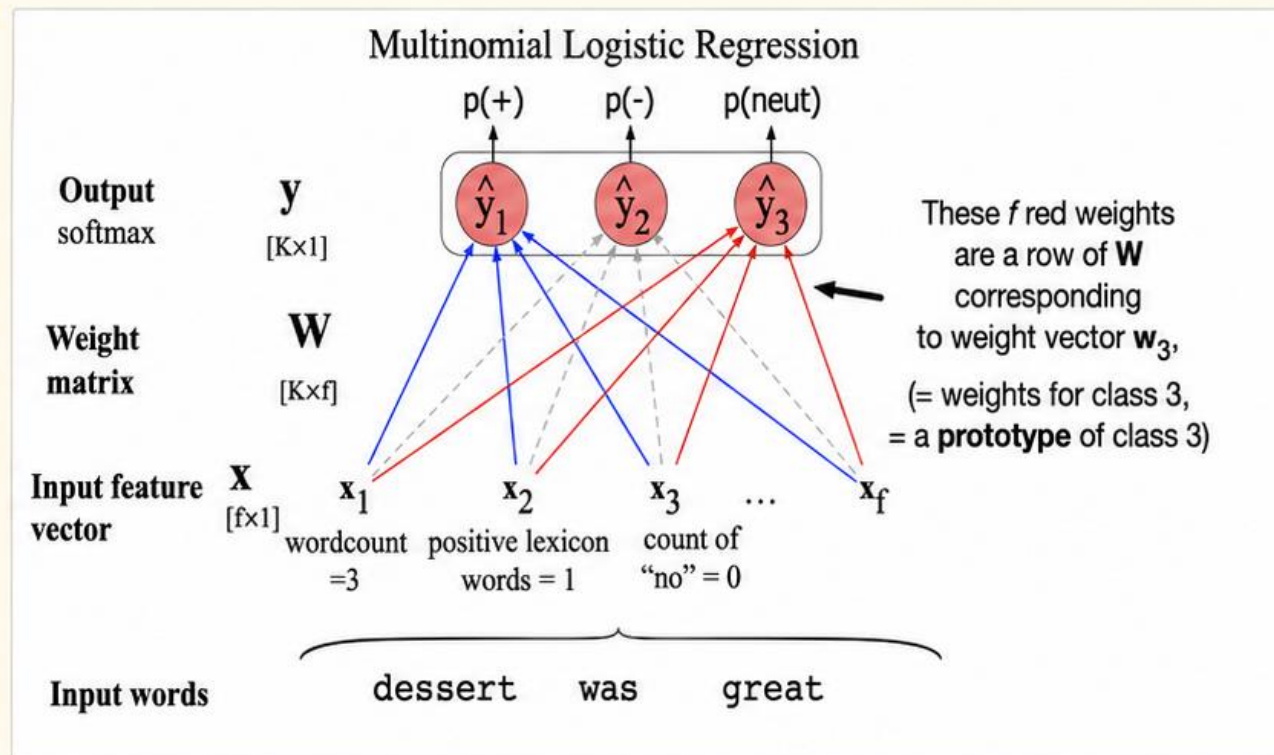
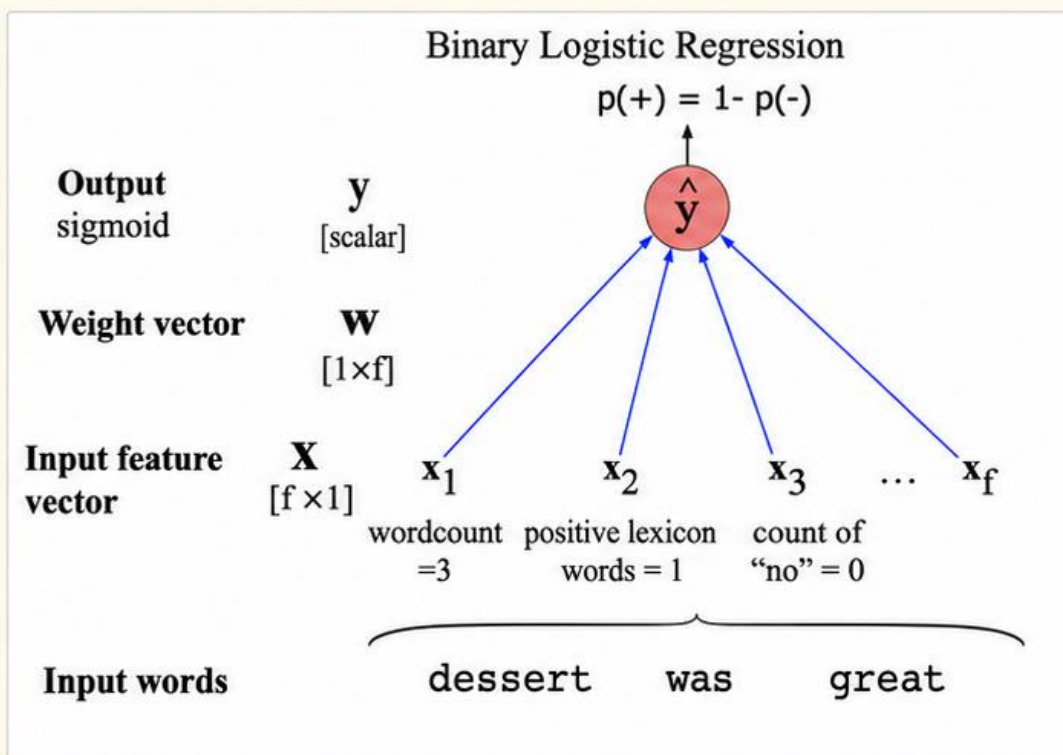
Target y is one-hot: the correct class has value 1; every other class has value 0.

Each row w_k of W is the class-specific weight vector—a learned prototype for class k .

How should learning penalize a model that assigns low probability to the correct class?

Binary and Multinomial Logistic Regression

Binary uses one weight vector; K-class classification packs K class-specific vectors into W .



Each row w_k of W is the weight vector for class k .

How can the same feature **support one class** and **oppose another**?

Vectorization Processes Many Examples at Once

Stack the m feature vectors as rows of X ; one matrix operation returns all m predictions.

One example at a time

$$\hat{y}^{(i)} = \sigma(\mathbf{w} \cdot \mathbf{x}^{(i)} + b), \quad i = 1, \dots, m$$

$$\hat{y}^{(1)} = \sigma(\mathbf{w} \cdot \mathbf{x}^{(1)} + b)$$

$$\hat{y}^{(2)} = \sigma(\mathbf{w} \cdot \mathbf{x}^{(2)} + b)$$

$$\hat{y}^{(3)} = \sigma(\mathbf{w} \cdot \mathbf{x}^{(3)} + b)$$

m repeated evaluations

STACK THE
ROWS



All examples in one operation

$$\mathbf{X} = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & \dots & x_f^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \dots & x_f^{(2)} \\ x_1^{(3)} & x_2^{(3)} & \dots & x_f^{(3)} \\ \vdots & \vdots & \ddots & \vdots \\ x_1^{(m)} & x_2^{(m)} & \dots & x_f^{(m)} \end{bmatrix} \in \mathbb{R}^{m \times f}$$

$$\hat{\mathbf{y}} = \sigma(\mathbf{X}\mathbf{w} + \mathbf{b})$$

$$(m \times 1) = \sigma((m \times f)(f \times 1) + (m \times 1))$$

σ is applied element-wise.

The matrix form is equivalent to the loop, but modern hardware evaluates it much more efficiently.

Once we can score a batch, how should we **learn w and b** ?

Accuracy Can Hide Failure on Imbalanced Classes

A classifier can look excellent by predicting only the majority class.

		gold standard labels		
		gold positive	gold negative	
system output labels	system positive	true positive	false positive	precision = $\frac{tp}{tp+fp}$
	system negative	false negative	true negative	
		recall = $\frac{tp}{tp+fn}$	accuracy = $\frac{tp+tn}{tp+fp+tn+fn}$	

The 'nothing is pie' classifier

1,000,000 tweets

100 about pie

999,900 unrelated

Prediction for every tweet: not about pie

$$\text{Accuracy} = \frac{999,900}{1,000,000} = 99.99\%$$

$$\text{Recall} = \frac{0}{100} = 0$$

It misses every customer comment we wanted to find.

Which measures focus on finding the **positive class**?

Precision and Recall Measure Different Errors

Precision checks positive predictions; recall checks the positive cases that truly exist.

Precision

$$P = \frac{TP}{TP + FP}$$

Of everything predicted positive,
how much was correct?

TP: correct detections

FP: false alarms

Recall

$$R = \frac{TP}{TP + FN}$$

Of everything truly positive,
how much did we find?

TP: found positives

FN: missed positives

Combine them with F-measure

$$F_{\beta} = \frac{(\beta^2 + 1)PR}{\beta^2 P + R}$$

$\beta > 1$ favors recall

$\beta < 1$ favors precision

$$F_1 = \frac{2PR}{P + R}$$

F_1 is the harmonic mean: it is pulled toward the smaller of P and R.

A high F_1 requires both precision and recall to be high.

How do we extend these measures **beyond two classes**?

Multiclass Evaluation Scores Each Class Separately

For class c , precision uses its prediction row; recall uses its gold-label column.

		<i>gold labels</i>			
		urgent	normal	spam	
<i>system output</i>	urgent	8	10	1	$\text{precision}_u = \frac{8}{8+10+1}$
	normal	5	60	50	$\text{precision}_n = \frac{60}{5+60+50}$
	spam	3	30	200	$\text{precision}_s = \frac{200}{3+30+200}$
		$\text{recall}_u = \frac{8}{8+5+3}$	$\text{recall}_n = \frac{60}{10+60+30}$	$\text{recall}_s = \frac{200}{1+50+200}$	

Read one class at a time

$$\text{Precision}_c = \frac{\text{diagonal entry}}{\text{row sum}}$$

$$\text{Recall}_c = \frac{\text{diagonal entry}}{\text{column sum}}$$

Urgent as the positive class

$$P_{\text{urgent}} = \frac{8}{8 + 10 + 1}$$

$$R_{\text{urgent}} = \frac{8}{8 + 5 + 3}$$

Repeat for normal and spam: each class gets its own precision and recall.

How should we **combine** the **class-specific scores**?

Macro and Micro Averaging Weight Classes Differently

Macro gives each class one vote; micro gives each decision one vote.

Class 1: Urgent

	true urgent	true not
system urgent	8	11
system not	8	340

$$\text{precision} = \frac{8}{8+11} = .42$$

Class 2: Normal

	true normal	true not
system normal	60	55
system not	40	212

$$\text{precision} = \frac{60}{60+55} = .52$$

Class 3: Spam

	true spam	true not
system spam	200	33
system not	51	83

$$\text{precision} = \frac{200}{200+33} = .86$$

Pooled

	true yes	true no
system yes	268	99
system no	99	635

$$\text{microaverage precision} = \frac{268}{268+99} = .73$$

$$\text{macroaverage precision} = \frac{.42 + .52 + .86}{3} = .60$$

Macroaverage

$$P_{\text{macro}} = \frac{(.42 + .52 + .86)}{3} = .60$$

Average the class-specific scores.

Each class contributes equally.

Use when performance on every class matters.

Microaverage

$$P_{\text{micro}} = \frac{268}{268 + 99} = .73$$

Pool all class decisions first.

Frequent classes contribute more decisions.

In this example, the spam class dominates.

For single-label multiclass classification: micro-precision = micro-recall.

How should we separate data for **training**, **model selection**, and **final evaluation**?

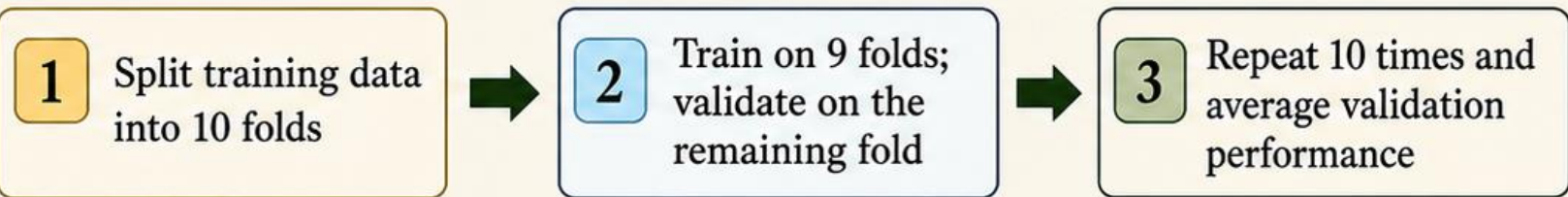
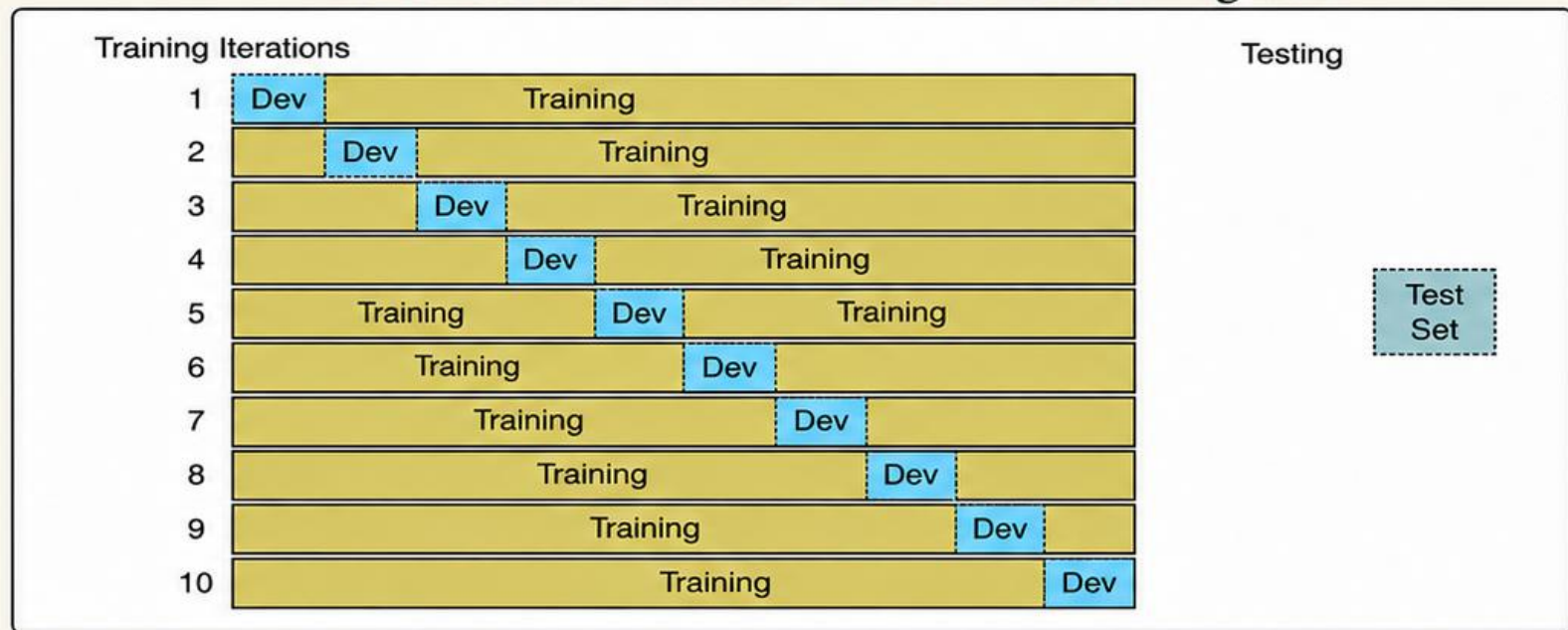
Cross-Validation Uses Training Data More Efficiently

Rotate the development fold inside the training data; reserve the final test set for one unbiased evaluation.

Three distinct roles

- 1 Training data** — fit model parameters
- 2 Development folds** — tune settings and select the model
- 3 Untouched test set** — report final performance once

10-fold cross-validation inside the training data



10 models; each trains on 90% of the training data.

Cross-validation supports model selection; the untouched test set supports the final performance claim.